

View-Bound Periodic Automorphism and Ring Action Specification

Stage 6A: Exact Validation Against PeriodicNetView Signatures

mdstats

2026-07-18

Contents

1	Purpose and gate status	2
2	Motive	2
3	Algorithmic provenance and attribution	3
4	Dependency boundary	3
5	Mathematical model	3
5.1	Lifted vertex action	3
5.2	Periodic-boundary subspace	4
5.3	Signature preservation	4
5.4	Explicit multiedge action	4
5.5	Physical edge-instance image	5
5.6	Mapping a primitive-ring placement	5
5.7	Ordered occurrence alignment	6
6	API standard	6
6.1	Exceptions	6
6.2	PeriodicEdgeImage	6
6.2.1	Meaning	7
6.2.2	Constraints	7
6.3	ValidatedPeriodicAutomorphism	7
6.3.1	Invariants	7
6.4	build_validated_periodic_automorphism	7
6.4.1	Input constraints	7
6.4.2	Output	8
6.4.3	Failure	8
6.5	map_lifted_vertex	8
6.6	map_lifted_edge_instance	8
6.6.1	Constraints	8
6.6.2	Output	8
6.7	RingOccurrenceMap	8
6.8	map_ring_placement	8
6.8.1	Input constraints	9
6.8.2	Output	9
6.8.3	Completeness semantics	9
7	Algorithm	9
8	Complexity	9

9	Edge cases and warnings	10
9.1	Same topology, different view	10
9.2	Ignored labels do not merge graph records	10
9.3	Dense edge domains are distinct	10
9.4	Partial periodicity	10
9.5	Common translation gauge	10
9.6	Catalog truncation	10
9.7	Repeated tokens and ring stabilizers	10
9.8	Direct dataclass construction	10
10	Focused verification	10
11	Deferred work	11
12	References	11

1 Purpose and gate status

This document specifies the view-bound periodic automorphism validation and primitive-ring action layer implemented in:

`mdstats/analysis/periodic_ring_action.py`

The package version for this runtime/API revision is:

`mdstats 0.19.16a0`

The module validates an explicitly supplied periodic multigraph automorphism against one immutable `PeriodicNetView`. It enforces:

1. exact lifted vertex action;
2. explicit multiedge permutation and orientation;
3. exact quotient-edge endpoint and lattice-shift incidence;
4. preservation of the active `NetViewPolicy` vertex and edge signatures;
5. compatibility with the view's periodic boundary-condition subspace; and
6. exact induced action on physical edge instances and translated primitive-ring occurrences.

This gate does **not** discover automorphisms, normalize representatives modulo a common lattice translation, compute a group, serialize a symmetry catalog, compute orbits/stabilizers, or derive a Euclidean space group. Those belong to later `net_symmetry.py` stages.

2 Motive

The Stage-5 P2 prototype proved that the current ring representation supports an exact periodic multigraph action. However, the prototype was bound only to a `PrimitiveRingIndex`. That is insufficient for final symmetry semantics because one `FrameworkTopology` may support several legitimate net interpretations.

For example, the same LTA framework may be viewed as:

- an unlabeled abstract T -net, where Si and Al may be exchanged; or
- a chemically decorated net, where Si and Al must remain distinct.

The connectivity is identical, but the automorphism groups can differ. Therefore an accepted action must belong to one exact

$$\text{Aut}(G, \sigma_V, \sigma_E),$$

where (σ_V, σ_E) are the deterministic signatures stored by `PeriodicNetView`.

The present module establishes that ownership and preserves the already validated occurrence-level ring action.

3 Algorithmic provenance and attribution

The periodic quotient representation follows the labelled finite-graph / edge-vector method of Chung, Hahn, and Klee [1]. A quotient edge carries an integer lattice translation and represents an infinite orbit of physical lifted edges.

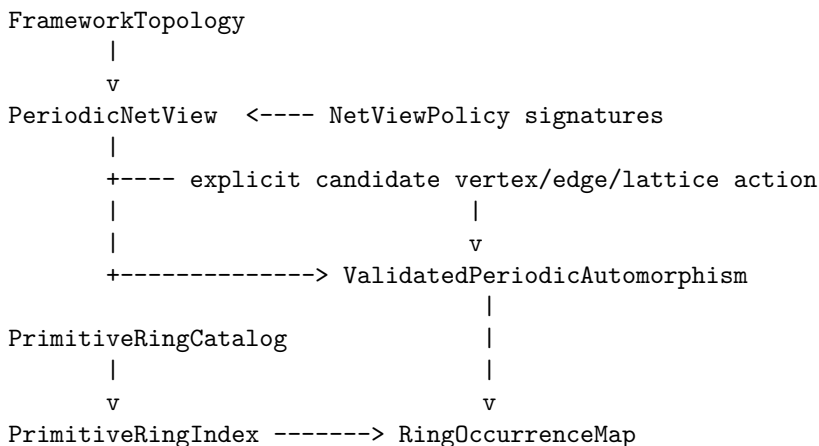
The exact combinatorial automorphism viewpoint follows Delgado-Friedrichs and O’Keeffe [2]. That work motivates computing periodic-net symmetry from exact combinatorial structure rather than fitting approximate space-group operations to a distorted geometric frame.

The following are mdstats-specific adaptations:

- ownership by `PeriodicNetView.digest`;
- deterministic signature-policy enforcement;
- explicit source-safe parallel-edge action;
- PBC-subspace compatibility checks for partially periodic graphs;
- conversion between net-view edge positions and primitive-ring catalog edge indices through stable `FrameworkEdgeKey`;
- exact lifted-edge-instance action; and
- exact ordered ring-occurrence alignment using stable ring keys, lifted vertices, and physical edge instances.

Implementation comments adjacent to the periodic incidence and signature checks must retain citations to [1,2].

4 Dependency boundary



`PeriodicNetView` owns the symmetry semantics. `PrimitiveRingIndex` is required only when inducing the accepted action on represented primitive rings.

5 Mathematical model

5.1 Lifted vertex action

A representative acts on a lifted quotient vertex as

$$g(i, \mathbf{n}) = (\pi_V(i), A\mathbf{n} + \tau_i),$$

where:

- π_V is a permutation of quotient framework vertices;
- $A \in GL(3, \mathbb{Z})$ is an integer unimodular matrix;
- $\tau_i \in \mathbb{Z}^3$ is the image shift of source base vertex i .

The implementation requires

$$\det A = \pm 1.$$

A common translation added to every τ_i changes only the selected representative of the same operation modulo lattice translations. This gate accepts such representatives but does not yet normalize their gauge.

5.2 Periodic-boundary subspace

For partial periodicity, valid image shifts have zero coordinates along nonperiodic axes. Therefore A must map the active translation subspace to itself. If P is the set of periodic axes and N the nonperiodic axes, then

$$A_{rc} = 0 \quad \text{for every } r \in N, c \in P.$$

This condition, together with full integer unimodularity, prevents a valid periodic translation from acquiring a nonperiodic component. Every supplied vertex image shift τ_i must also vanish along nonperiodic axes; otherwise the record does not represent a lattice image of the partially periodic net.

5.3 Signature preservation

Let the view assign deterministic signatures

$$\sigma_V : V \rightarrow \mathcal{S}_V, \quad \sigma_E : E \rightarrow \mathcal{S}_E.$$

A valid view automorphism must satisfy

$$\sigma_V(\pi_V(i)) = \sigma_V(i)$$

for every vertex, and

$$\sigma_E(\pi_E(e)) = \sigma_E(e)$$

for every edge orbit.

Ignoring a decoration in `NetViewPolicy` may make two signatures equal and permit exchange. It never merges the underlying graph records.

5.4 Explicit multiedge action

A source quotient edge is

$$e = (i, j, \Delta).$$

Its canonical physical instance anchored at $\mathbf{a}$ joins

$$(i, \mathbf{a}) \longrightarrow (j, \mathbf{a} + \Delta).$$

A vertex permutation alone is insufficient in a multigraph. Each source edge position therefore stores an explicit target edge position and orientation

$$s_e \in \{+1, -1\}.$$

For target edge

$$e' = (u, v, \Delta'),$$

forward orientation requires

$$\pi_V(i) = u, \quad \pi_V(j) = v,$$

and

$$A\Delta + \tau_j = \tau_i + \Delta'.$$

Reverse orientation requires

$$\pi_V(i) = v, \quad \pi_V(j) = u,$$

and

$$A\Delta + \tau_j = \tau_i - \Delta'.$$

All equations are checked using exact integer arithmetic.

5.5 Physical edge-instance image

For source physical instance $(e, \mathbf{a})$:

- if $s_e = +1$,

$$\mathbf{a}' = A\mathbf{a} + \tau_i;$$

- if $s_e = -1$,

$$\mathbf{a}' = A\mathbf{a} + A\Delta + \tau_j.$$

The result is the exact target physical edge instance $(e', \mathbf{a}')$.

5.6 Mapping a primitive-ring placement

A source placement is

$$R(q, \mathbf{t}) = \widehat{R}(q) + \mathbf{t},$$

where q is a stable `PrimitiveRingKey` and $\widehat{R}(q)$ is the stored canonical lifted representative.

The ring step edge index belongs to `PrimitiveRingCatalog`; the automorphism edge position belongs to `PeriodicNetView`. These dense domains are not assumed to share an ordering. Conversion is performed through stable `FrameworkEdgeKey`:

```
ring edge index
-> FrameworkEdgeKey
-> net-view source edge position
-> target net-view edge position
-> target FrameworkEdgeKey
-> ring edge index
```

Each source ring step (e_k, ϵ_k) maps to

$$\left(\pi_E(e_k), \epsilon_k s_{e_k}\right).$$

The transformed token sequence identifies the target stable ring key. It does not choose the ordered occurrence alignment.

5.7 Ordered occurrence alignment

For an n -ring, every cyclic/reversed parameterization is considered. For start position c and orientation $\epsilon \in \{+1, -1\}$,

$$p_V(k) = c + \epsilon k \pmod{n},$$

and

$$p_E(k) = \begin{cases} c + k \pmod{n}, & \epsilon = +1, \\ c - k - 1 \pmod{n}, & \epsilon = -1. \end{cases}$$

A parameterization is accepted only when:

1. all mapped lifted vertices equal target canonical vertices plus one common lattice translation;
2. all mapped steps have the required target edge and traversal orientation; and
3. every mapped physical lifted edge instance equals the aligned translated target instance.

Exactly one alignment must survive.

6 API standard

The API is supported under `mdstats.analysis`. Group-level symmetry APIs remain provisional until automatic discovery and group closure are implemented.

6.1 Exceptions

```
class PeriodicRingActionError(ValueError):
    """Base exception."""

class PeriodicRingActionInputError(PeriodicRingActionError):
    """Malformed input or source/view mismatch."""

class PeriodicRingActionValidationError(PeriodicRingActionError):
    """Signature, PBC-subspace, or exact incidence violation."""

class RingOccurrenceMappingError(PeriodicRingActionError):
    """No unique represented target ring occurrence can be recovered."""
```

6.2 PeriodicEdgeImage

```
@dataclass(frozen=True, order=True, slots=True)
class PeriodicEdgeImage:
    target_edge_index: int
    orientation: Literal[-1, 1]

    @property
    def target_edge_position(self) -> int: ...
```

6.2.1 Meaning

`target_edge_index` is a dense position in the owning `PeriodicNetView.edge_keys` sequence. The historical field name is retained for compatibility; the `target_edge_position` property states the intended semantics.

6.2.2 Constraints

- `target_edge_index >= 0`;
- `orientation` is exactly +1 or -1;
- the complete `edge_images` tuple forms a permutation of all source view edge positions.

6.3 ValidatedPeriodicAutomorphism

```
@dataclass(frozen=True, slots=True)
class ValidatedPeriodicAutomorphism:
    periodic_net_view_digest: str
    topology_graph_digest: str
    lattice_matrix: tuple[tuple[int, int, int], ...]
    vertex_atom_indices: tuple[int, ...]
    vertex_images: tuple[LiftedVertexRef, ...]
    edge_images: tuple[PeriodicEdgeImage, ...]

    def vertex_image(self, atom_index: int) -> LiftedVertexRef: ...
```

6.3.1 Invariants

- both digests are valid SHA-256 strings;
- `lattice_matrix` is integer and unimodular;
- `vertex_atom_indices` equals the owning view's sorted vertex sequence;
- `vertex_images` is a permutation of the same vertex set;
- `edge_images` is a permutation of the owning view edge positions;
- all signatures and exact incidence equations were validated by the builder.

Direct dataclass construction checks local structure but cannot prove relation to a source view. Scientific callers must use the builder.

6.4 build_validated_periodic_automorphism

```
def build_validated_periodic_automorphism(
    view: PeriodicNetView,
    *,
    lattice_matrix: Sequence[Sequence[int]],
    vertex_images: Mapping[int, LiftedVertexRef],
    edge_images: Sequence[PeriodicEdgeImage],
) -> ValidatedPeriodicAutomorphism:
    ...
```

6.4.1 Input constraints

- `view` is an immutable `PeriodicNetView`;
- every source view vertex appears exactly once in `vertex_images`;
- target vertex atom indices form a permutation of the source set;
- `edge_images` has exactly `view.n_edges` entries and forms a target permutation;
- A is unimodular and preserves the active PBC subspace;
- mapped vertex and edge signatures match source signatures;
- mapped quotient-edge endpoints and lattice shifts satisfy the exact forward or reverse incidence equations.

6.4.2 Output

A view-bound immutable validated action.

6.4.3 Failure

- malformed values: `PeriodicRingActionInputError`;
- invalid signature, PBC, or incidence action: `PeriodicRingActionValidationError`.

6.5 map_lifted_vertex

```
def map_lifted_vertex(  
    automorphism: ValidatedPeriodicAutomorphism,  
    vertex: LiftedVertexRef,  
) -> LiftedVertexRef:  
    ...
```

Returns the exact lifted image under the already validated action.

6.6 map_lifted_edge_instance

```
def map_lifted_edge_instance(  
    view: PeriodicNetView,  
    automorphism: ValidatedPeriodicAutomorphism,  
    edge_instance: LiftedEdgeInstanceRef,  
) -> LiftedEdgeInstanceRef:  
    ...
```

6.6.1 Constraints

- `automorphism.periodic_net_view_digest == view.digest`;
- topology graph digests agree;
- the edge key is present in the view.

6.6.2 Output

The exact target source-bound physical edge instance.

6.7 RingOccurrenceMap

```
@dataclass(frozen=True, slots=True)  
class RingOccurrenceMap:  
    periodic_net_view_digest: str  
    topology_graph_digest: str  
    source_placement: RingPlacement  
    target_placement: RingPlacement  
    source_vertex_position_to_target_position: tuple[int, ...]  
    source_step_position_to_target_position: tuple[int, ...]  
    parameterization: CycleParameterization
```

The explicit permutations are authoritative. `parameterization` must reproduce both exactly.

6.8 map_ring_placement

```
def map_ring_placement(  
    index: PrimitiveRingIndex,  
    view: PeriodicNetView,  
    automorphism: ValidatedPeriodicAutomorphism,  
    placement: RingPlacement,
```



```
) -> RingOccurrenceMap:
...
```

6.8.1 Input constraints

- index and view share `source_graph_digest`;
- index and view expose the same stable framework-edge-key set;
- automorphism belongs to the exact view digest;
- placement belongs to the same topology graph;
- the transformed target ring exists in the represented catalog.

6.8.2 Output

One exact target placement and ordered occurrence map.

6.8.3 Completeness semantics

For a structurally valid action and a represented target ring, the routine tests all $2n$ cyclic/reversed parameterizations and is exhaustive for the stored canonical ring boundaries.

Failure to find a target ring can mean the ring catalog is truncated or incomplete; it is not evidence that the proposed graph action is globally invalid.

7 Algorithm

```
validate candidate action against one PeriodicNetView
|
+-- check vertex permutation
+-- check edge permutation
+-- check integer unimodular lattice action
+-- check PBC subspace
+-- check vertex signatures
+-- check edge signatures
+-- check exact endpoint/image-shift incidence
v
ValidatedPeriodicAutomorphism
|
+-- map lifted vertices directly
+-- map physical edge instances through view edge positions
+-- bridge view/ring edge domains through FrameworkEdgeKey
+-- transform ordered ring steps
+-- identify target PrimitiveRingKey
+-- exhaust 2n cyclic/reversed alignments
+-- verify exact lifted vertices and edge instances
v
RingOccurrenceMap
```

8 Complexity

Let $|V|$ and $|E|$ be the view sizes.

Action validation costs

$$O(|V| + |E|).$$

For an n -ring, the current transparent alignment implementation tests $2n$ parameterizations with $O(n)$ vertex/step/edge checks each:

$$O(n^2).$$

Primitive rings are short in the intended framework applications. A linear-time cyclic matcher is unnecessary until profiling demonstrates a bottleneck.

9 Edge cases and warnings

9.1 Same topology, different view

An action validated under one view digest must be rejected under every other view, even if their current signatures happen to produce the same permutation set. Policy provenance is part of the scientific result.

9.2 Ignored labels do not merge graph records

Equal signatures allow exchange only. Parallel edge multiplicity and stable edge keys remain exact.

9.3 Dense edge domains are distinct

`PeriodicEdgeImage.target_edge_index` belongs to the view. `PrimitiveRingStep.edge_index` belongs to the ring catalog. Never compare them directly; convert through `FrameworkEdgeKey`.

9.4 Partial periodicity

A matrix that sends an active periodic translation into a nonperiodic coordinate is invalid even if its full determinant is ± 1 .

9.5 Common translation gauge

Adding one common shift to every vertex image gives a translated representative. This gate preserves it. Later symmetry catalogs must choose a deterministic gauge before group equality, composition, or serialization.

9.6 Catalog truncation

An exact net automorphism may map a ring outside a bounded represented ring family. `RingOccurrenceMappingError` then reports an unavailable target; it does not invalidate the net action.

9.7 Repeated tokens and ring stabilizers

Token canonicalization identifies a target key but cannot select an occurrence alignment. Exact lifted vertices and physical edge instances remain authoritative.

9.8 Direct dataclass construction

Constructing `ValidatedPeriodicAutomorphism` directly checks only local field invariants. It does not establish signature or incidence validity. Use the builder.

10 Focused verification

The gate requires tests for:

1. identity and common-translation representatives;
2. nontrivial unimodular lattice action;
3. cyclic rotation and reversed boundary orientation;
4. explicit parallel-edge exchange;
5. invalid structural edge action;
6. topology and view-digest mismatch rejection;

7. an Si/Al exchange accepted by the unlabeled view and rejected by the chemical view;
8. an O/S parallel-edge exchange accepted by the unlabeled view and rejected by the chemical view;
9. active-PBC-subspace rejection;
10. Na-LTA mapping of all 82 ring orbits and 432 ordered ring-step occurrences.

The focused Stage-4/5/net-view gate for `mdstats 0.19.15a0` passes:

88 passed

11 Deferred work

The following are explicitly outside this specification:

- automatic periodic-net automorphism discovery;
- deterministic representative gauge normalization;
- composition, inverse, and group closure;
- finite representatives modulo translation;
- vertex, edge, and ring orbit catalogs;
- ring stabilizers;
- symmetry serialization;
- barycentric/crystallographic realization;
- embedded-face and tile action.

12 References

- [1] S. J. Chung, Th. Hahn, and W. E. Klee, “Nomenclature and generation of three-periodic nets: the vector method,” *Acta Crystallographica A* **40**, 42–50 (1984). DOI: 10.1107/S0108767384000088.
- [2] O. Delgado-Friedrichs and M. O’Keeffe, “Identification of and symmetry computation for crystal nets,” *Acta Crystallographica A* **59**, 351–360 (2003). DOI: 10.1107/S0108767303012017.